BASES DE DATOS NOSQL · PRÁCTICA GUIADA 6 · ORIENTADA A COLUMNAS · GUÍA CON SOLUCIÓN COMPLETA

Estación meteorológica UTHH: una base de datos de columnas en la nube, construida desde cero

Desde un repositorio vacío: levanta tu propio servidor de Apache Cassandra en GitHub Codespaces, diseña las tablas a partir de las consultas, manipula los datos con CQL y construye una aplicación web que recibe miles de lecturas de sensores, también desde tu celular.

Equipo: _____ Grupo: _____

Integrantes: _____ Fecha: _____

¿Qué vas a construir?

La UTHH instala tres sensores (Biblioteca, Cafetería y Explanada) que miden temperatura y humedad **cada segundo**. En un año son casi 95 millones de lecturas. Ese volumen de escrituras constantes, que se consultan por sensor y por fecha, es el caso típico de una base de datos **orientada a columnas** (familias de columnas). Vas a construir todo tú, sin plantilla:

- El **servidor de base de datos**: Apache Cassandra en un contenedor dentro de tu codespace.
- El **keyspace y las tablas**, diseñadas a partir de las preguntas que hará la aplicación.
- Un **simulador de sensores** y una carga histórica de miles de lecturas.
- Una **API REST** en Node.js y una **aplicación web**: tablero en tiempo real, gráfica por sensor y registro de lecturas desde el celular con un QR.

Sensores (simulador) y celular → API en Node.js → CQL → Cassandra en Docker → Particiones en disco

Objetivos de aprendizaje

- Implementar un gestor no relacional en la nube aprovisionando tu propio servidor de Cassandra.
- Explicar keyspace, partición, llave de partición, columna de agrupamiento y desnormalización.
- Diseñar tablas a partir de las consultas (al revés que en MySQL) y crearlas con CQL.
- Insertar, consultar, actualizar y borrar datos con CQL, incluidos upsert, TTL, contadores y BATCH.
- Conectar una aplicación web a Cassandra y observar su rendimiento de escritura.
- Reconocer los límites del modelo (sin JOIN, consultas solo por llave) y sus riesgos de seguridad.

Lo que necesitas

- Una cuenta de GitHub (de preferencia con tu correo institucional), un navegador actualizado y un celular.
- Esta guía: contiene **todo** el código. No hay plantilla: cada archivo lo creas tú.

Antes de empezar

Escribe los comandos **una línea a la vez**. En la terminal, lo que va después de `#` es comentario; en CQL el comentario es `--`; en JavaScript, `//`. Lee los comentarios: explican qué hace cada línea. Usa comillas rectas (`'` y `"`), no tipográficas.

Parte 0 · Primero las preguntas, después las tablas

En MySQL diseñas las tablas según las **entidades** (sensor, lectura), normalizas y después escribes las consultas con JOIN. En Cassandra es al revés: **primero escribes las preguntas** que hará la aplicación y diseñas **una tabla por pregunta**. Así cada consulta lee un solo lugar del disco, aunque haya millones de filas y muchos servidores.

Conceptos de Cassandra

Concepto	Qué es	En esta práctica
Nodo y **clúster**	Un nodo es un servidor de Cassandra; varios nodos forman un clúster y se reparten los datos.	Un nodo en Docker
Keyspace	Contenedor de tablas con su regla de réplica (cuántas copias de cada dato).	`estacion`
Partición	Grupo de filas que se guardan juntas en el mismo nodo. Es la unidad de reparto.	Las lecturas de un sensor en un día
Llave de partición	Columnas que deciden en qué partición (y en qué nodo) vive cada fila. Toda consulta debe darla.	`(sensor, dia)`
Columna de agrupamiento (clustering)	Ordena las filas dentro de la partición; permite rangos y orden.	`hora`, de la más nueva a la más vieja
Desnormalización	Guardar el mismo dato en varias tablas, una por consulta. No hay JOIN.	Una lectura se escribe en 3 o 4 tablas
Upsert	INSERT y UPDATE hacen lo mismo: si la llave existe, sobrescriben.	`ultima_lectura`
CQL	Cassandra Query Language: se parece a SQL, pero sin JOIN y con reglas por llave.	Toda la práctica

Las preguntas de la aplicación y sus tablas

#	Pregunta	Tabla	Llave primaria
1	¿Qué sensores hay y cuál es su umbral de alerta?	`sensores`	`(id)`
2	¿Cómo cambió la temperatura de un sensor en un día?	`lecturas_por_sensor_dia`	`((sensor, dia), hora)`
3	¿Cuál es la última lectura de cada sensor?	`ultima_lectura`	`(sensor)`
4	¿Qué alertas hubo en un día?	`alertas_por_dia`	`((dia), hora, sensor)`
5	¿Cuántas lecturas lleva cada sensor por día?	`conteo_por_dia`	`((sensor), dia)`

Cómo se ve una partición

```
Tabla lecturas_por_sensor_dia
┌ partición ('uthh-03', 2026-10-16) ──────────────────────────┐
│ hora 15:02:07  temperatura 35.8  humedad 60   ← la más reciente│
│ hora 15:02:06  temperatura 35.7  humedad 61                  │
│ hora 15:02:05  temperatura 35.7  humedad 61   … miles de filas │
└──────────────────────────────────────────────────────────────┘

┌ partición ('uthh-01', 2026-10-16) ──────────────────────────┐
│ hora 15:02:07  temperatura 28.1  humedad 68                  │
└──────────────────────────────────────────────────────────────┘
Cada partición puede vivir en un nodo distinto del clúster.
```

Tabla espejo: MySQL ↔ Cassandra ↔ IRIS

Idea	MySQL	Cassandra	InterSystems IRIS
Crear la base	`CREATE DATABASE`	`CREATE KEYSPACE` + réplica	Base de datos + namespace
Estructura	Tablas normalizadas	Una tabla por consulta	Clases persistentes
Llave	PRIMARY KEY	Llave de partición + agrupamiento	ID del objeto
Relaciones	FOREIGN KEY y JOIN	No hay: se duplican datos	Referencias y embebidos
Consultar	Cualquier columna	Siempre con la llave de partición	Objetos o SQL
Escalar	Servidor más grande	Más nodos	Servidor más grande o fragmentación
Fuerte en	Transacciones	Escrituras masivas y series de tiempo	Reglas de negocio

Para comentar en clase

En MySQL tendrías **una** tabla `lectura` y harías `SELECT ... WHERE sensor_id = 1 AND fecha = '2026-10-16'`. Funciona con miles de filas; con cientos de millones en varios servidores, ese `WHERE` obligaría a revisar todos. Cassandra te obliga a decidir desde el diseño en qué partición está la respuesta.

Parte 1 · Tu propio espacio en la nube

1 Crear tu repositorio vacío

1. En **github.com**: botón **+ → New repository**.
2. **Repository name:** `estacion-cassandra`. Marca **Add a README file**. Puede ser público o privado.
3. Presiona **Create repository**.

Tu repositorio solo tiene un `README.md`. Todo lo demás lo vas a crear tú.

2 Crear el codespace

1. En tu repositorio: **Code → Codespaces → Create codespace on main**.
2. Se abre VS Code en el navegador, con una **Terminal** abajo (si no aparece: ☰ → **Terminal → Nueva terminal**).

El codespace es una máquina Linux en la nube de GitHub. La imagen que usa por defecto ya trae **Docker** (para correr servidores en contenedores) y **Node.js** (para la aplicación). Compruébalo:

```
docker --version          # version de Docker instalada
node --version            # version de Node.js (debe ser 18 o mayor)
```

Mantén vivo tu codespace

GitHub lo apaga tras **30 minutos sin actividad en VS Code**. Si se apaga, ábrelo desde **github.com/codespaces**; tus archivos se conservan y en la Parte 2 verás cómo volver a encender Cassandra.

Parte 2 · Aprovisionar tu servidor de Cassandra (implementar el gestor)

3 Levantar Cassandra en un contenedor

Con Firestore, Google te dio la base de datos ya funcionando (servicio administrado). Aquí **tú** levantas el servidor, como lo haría un administrador en la nube:

```
docker run -d --name cassandra -p 9042:9042 -e MAX_HEAP_SIZE=1G -e HEAP_NEWSIZE=256M --
restart unless-stopped cassandra:5.0
```

Parte del comando	Qué hace
`docker run -d`	Crea y arranca un contenedor en segundo plano.
`--name cassandra`	Le pone nombre para referirte a él después.

`-p 9042:9042`	Publica el puerto 9042, por donde los programas hablan CQL con Cassandra.
`-e MAX_HEAP_SIZE=1G -e HEAP_NEWSIZE=256M`	Limita la memoria de Java para que quepa en el codespace gratuito.
`--restart unless-stopped`	Lo vuelve a encender solo cuando el codespace se reinicia.
`cassandra:5.0`	La imagen oficial de Apache Cassandra 5.0 (la primera vez se descarga).

```
docker ps                      # lista los contenedores: debe aparecer "cassandra" con
STATUS "Up"
docker logs -f cassandra       # mira el arranque; espera la linea "Starting listening for
CQL clients"
```

Para salir de los logs presiona **Ctrl + C** (el servidor sigue corriendo). El primer arranque tarda de 1 a 2 minutos.

```
docker exec cassandra nodetool status     # estado del cluster: el nodo debe decir UN (Up /
Normal)
```

Resultado esperado

Una línea que empieza con `UN` y la dirección del nodo, en el centro de datos `datacenter1`. Tienes un clúster de Cassandra de **un nodo**.

Evidencia 1

Captura de `docker ps` y de `nodetool status` con el nodo en `UN`.

4 Entrar a cqlsh, la terminal de Cassandra

```
docker exec -it cassandra cqlsh          # abre cqlsh dentro del contenedor
```

El prompt cambia a `cqlsh>`. Ahí se escribe CQL, y cada sentencia termina en `;`:

```
SHOW VERSION;                             -- version de Cassandra y de CQL
DESCRIBE KEYSPACES;                       -- keyspaces que ya existen (todos son del
sistema)
SELECT cluster_name, data_center FROM system.local;   -- datos de este nodo
```

Para salir de cqlsh escribe `exit`.

Si al volver a abrir el codespace Cassandra no responde

Ejecuta `docker ps -a`. Si el contenedor dice `Exited`, enciéndelo con `docker start cassandra` y espera un minuto. Tus datos siguen ahí.

Parte 3 · Crear la base de datos con CQL (el CREATE DATABASE y los CREATE TABLE)

5 Crear el keyspace

Dentro de `cqlsh>`:

```
CREATE KEYSPACE IF NOT EXISTS estacion
  WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};   -- 1 copia: tenemos 1 nodo
DESCRIBE KEYSPACES;                          -- ya aparece estacion
USE estacion;                                -- trabajar dentro del keyspace (como USE en MySQL)
```

¿Qué es replication_factor?

Cuántas copias de cada partición guarda el clúster. En producción se usa 3: si un nodo falla, otros dos tienen el dato. Con un solo nodo solo cabe 1.

6 Crear las cinco tablas

Escribe cada `CREATE TABLE` en `cqlsh` (puedes escribirlo en varias líneas; se ejecuta al llegar al `;`). Después de cada una, revísala con `DESCRIBE TABLE nombre;`.

```
CREATE TABLE sensores (
  id        text PRIMARY KEY,      -- un sensor por particion
  nombre    text,
  ubicacion text,
  tipo      text,                  -- interior o exterior
  temp_max  float                  -- umbral de alerta en grados C
);
```

```
CREATE TABLE lecturas_por_sensor_dia (
  sensor      text,
  dia         date,
  hora        timestamp,
  temperatura float,
  humedad     int,
  PRIMARY KEY ((sensor, dia), hora)          -- particion (sensor, dia); dentro, filas por hora
) WITH CLUSTERING ORDER BY (hora DESC);      -- la lectura mas reciente primero
```

```
CREATE TABLE ultima_lectura (
  sensor      text PRIMARY KEY,    -- un INSERT con el mismo sensor sobrescribe (upsert)
  hora        timestamp,
  temperatura float,
  humedad     int
);
```

```
CREATE TABLE alertas_por_dia (
  dia         date,
  hora        timestamp,
  sensor      text,
  temperatura float,
  mensaje     text,
  PRIMARY KEY ((dia), hora, sensor)          -- particion: el dia
) WITH CLUSTERING ORDER BY (hora DESC, sensor ASC);
```

```
CREATE TABLE conteo_por_dia (
  sensor   text,
  dia      date,
  lecturas counter,                -- contador: solo se suma o se resta
  PRIMARY KEY ((sensor), dia)
) WITH CLUSTERING ORDER BY (dia DESC);
```

```
DESCRIBE TABLES;                          -- deben aparecer las 5
DESCRIBE TABLE lecturas_por_sensor_dia;    -- la definicion completa con sus opciones
```

- Doble paréntesis `((sensor, dia), hora)`: lo de adentro es la **llave de partición** (compuesta); lo que sigue, la **columna de agrupamiento**.
- Una partición por sensor y por día evita particiones gigantes: un sensor que escribe cada segundo genera 86,400 filas al día; sin el día, crecería sin límite.
- `counter` no puede mezclarse con columnas normales: por eso el conteo tiene su propia tabla.

Si te equivocas en una tabla

Bórrala y vuelve a crearla: `DROP TABLE nombre;`. En la Parte 3 todavía no hay datos que perder.

7 Guardar el esquema en tu repositorio

Lo que escribiste en cqlsh no queda en tu repositorio. Guárdalo como archivo para poder recrear la base en otro servidor:

1. En el Explorador de VS Code crea la carpeta `cql` y dentro el archivo `esquema.cql`.
2. Escribe en él el esquema completo (es lo mismo que acabas de ejecutar, más los sensores de la Parte 4):

`cql/esquema.cql`

```
-- Estacion UTHH · esquema de la base de datos en Apache Cassandra
-- En Cassandra primero se piensa en las CONSULTAS y despues en las tablas:
-- cada tabla responde una pregunta de la aplicacion.

-- Keyspace: el contenedor de las tablas (como la "base de datos" en MySQL).
-- replication_factor 1: una sola copia, porque tenemos un solo nodo.
CREATE KEYSPACE IF NOT EXISTS estacion
  WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};

USE estacion;

-- Catalogo de sensores. Pregunta: "¿que sensores hay y cual es su umbral?"
CREATE TABLE IF NOT EXISTS sensores (
  id        text PRIMARY KEY,   -- llave de particion: un sensor por particion
  nombre    text,
  ubicacion text,
  tipo      text,               -- interior o exterior
  temp_max  float               -- umbral de alerta en °C
);

-- Historial. Pregunta: "¿como cambio la temperatura de UN sensor en UN dia?"
CREATE TABLE IF NOT EXISTS lecturas_por_sensor_dia (
  sensor      text,
  dia         date,
  hora        timestamp,
  temperatura float,
  humedad     int,
  PRIMARY KEY ((sensor, dia), hora)   -- particion (sensor, dia); dentro, ordenadas por hora
) WITH CLUSTERING ORDER BY (hora DESC);   -- la mas reciente primero

-- Tablero. Pregunta: "¿cual es la ultima lectura de cada sensor?"
CREATE TABLE IF NOT EXISTS ultima_lectura (
  sensor      text PRIMARY KEY,   -- cada INSERT con el mismo sensor SOBRESCRIBE (upsert)
  hora        timestamp,
  temperatura float,
  humedad     int
);

-- Alertas. Pregunta: "¿que alertas hubo en un dia?"
CREATE TABLE IF NOT EXISTS alertas_por_dia (
  dia         date,
  hora        timestamp,
  sensor      text,
  temperatura float,
  mensaje     text,
  PRIMARY KEY ((dia), hora, sensor)   -- particion: el dia
) WITH CLUSTERING ORDER BY (hora DESC, sensor ASC);

-- Contador. Pregunta: "¿cuantas lecturas lleva cada sensor por dia?"
CREATE TABLE IF NOT EXISTS conteo_por_dia (
  sensor   text,
  dia      date,
  lecturas counter,                   -- tipo especial: solo se suma o se resta
  PRIMARY KEY ((sensor), dia)
) WITH CLUSTERING ORDER BY (dia DESC);

-- Los tres sensores de la escuela
INSERT INTO sensores (id, nombre, ubicacion, tipo, temp_max) VALUES ('uthh-01', 'Biblioteca',
'Edificio A, planta alta', 'interior', 30);
INSERT INTO sensores (id, nombre, ubicacion, tipo, temp_max) VALUES ('uthh-02', 'Cafeteria',
'Edificio C', 'interior', 32);
INSERT INTO sensores (id, nombre, ubicacion, tipo, temp_max) VALUES ('uthh-03', 'Explanada', 'Patio
central', 'exterior', 35);
```

Con este archivo, cualquier servidor nuevo se prepara con un solo comando (desde la terminal normal, fuera de cqlsh):

```
docker exec -i cassandra cqlsh < cql/esquema.cql     # ejecuta todo el archivo en Cassandra
```

Evidencia 2

Captura de `DESCRIBE TABLES;` con las 5 tablas y de `DESCRIBE TABLE lecturas_por_sensor_dia;`.

Parte 4 · Manipular datos con CQL

8 Insertar y consultar los sensores

En `cqlsh`, dentro de `estacion` (si saliste: `docker exec -it cassandra cqlsh` y `USE estacion;`):

```
INSERT INTO sensores (id, nombre, ubicacion, tipo, temp_max) VALUES ('uthh-01',
'Biblioteca', 'Edificio A, planta alta', 'interior', 30);
INSERT INTO sensores (id, nombre, ubicacion, tipo, temp_max) VALUES ('uthh-02',
'Cafeteria', 'Edificio C', 'interior', 32);
INSERT INTO sensores (id, nombre, ubicacion, tipo, temp_max) VALUES ('uthh-03',
'Explanada', 'Patio central', 'exterior', 35);
SELECT * FROM sensores;                                  -- los 3 sensores
SELECT nombre, temp_max FROM sensores WHERE id = 'uthh-03';   -- por su llave: directo a la
particion
```

9 Insertar lecturas y consultar una partición

```
INSERT INTO lecturas_por_sensor_dia (sensor, dia, hora, temperatura, humedad) VALUES
('uthh-01', '2026-10-16', '2026-10-16 09:00:00-0600', 24.5, 80);
INSERT INTO lecturas_por_sensor_dia (sensor, dia, hora, temperatura, humedad) VALUES
('uthh-01', '2026-10-16', '2026-10-16 12:00:00-0600', 27.8, 70);
INSERT INTO lecturas_por_sensor_dia (sensor, dia, hora, temperatura, humedad) VALUES
('uthh-01', '2026-10-16', '2026-10-16 15:00:00-0600', 30.6, 62);
INSERT INTO lecturas_por_sensor_dia (sensor, dia, hora, temperatura, humedad) VALUES
('uthh-01', '2026-10-17', '2026-10-17 09:00:00-0600', 23.9, 82);
```

```
SELECT * FROM lecturas_por_sensor_dia WHERE sensor = 'uthh-01' AND dia = '2026-10-16';
-- una particion: 3 filas, la mas nueva primero
SELECT hora, temperatura FROM lecturas_por_sensor_dia WHERE sensor = 'uthh-01' AND dia =
'2026-10-16' LIMIT 1;   -- solo la ultima
SELECT hora, temperatura FROM lecturas_por_sensor_dia WHERE sensor = 'uthh-01' AND dia =
'2026-10-16' AND hora >= '2026-10-16 12:00:00-0600';   -- rango por la columna de
agrupamiento
SELECT sensor, dia, token(sensor, dia) FROM lecturas_por_sensor_dia;   -- el token decide
en que nodo vive cada particion
```

- `-0600` es la zona horaria de México: Cassandra guarda la hora en UTC.
- El rango sobre `hora` funciona porque es columna de agrupamiento y las filas ya están ordenadas en disco.
- Los dos días de `uthh-01` tienen **tokens distintos**: son particiones distintas y en un clúster podrían estar en nodos distintos.

10 Upsert, UPDATE, DELETE, TTL, contadores y BATCH

Upsert: no existe el error de llave duplicada; un INSERT con la misma llave sobrescribe.

```
INSERT INTO ultima_lectura (sensor, hora, temperatura, humedad) VALUES ('uthh-01',
toTimestamp(now()), 27.0, 71);
INSERT INTO ultima_lectura (sensor, hora, temperatura, humedad) VALUES ('uthh-01',
toTimestamp(now()), 27.4, 70);
SELECT * FROM ultima_lectura;              -- una sola fila para uthh-01, con 27.4
```

UPDATE y DELETE (siempre con la llave completa):

```
UPDATE sensores SET temp_max = 31 WHERE id = 'uthh-01';                -- cambia el umbral
DELETE FROM lecturas_por_sensor_dia WHERE sensor = 'uthh-01' AND dia = '2026-10-17';   --
borra una particion completa
```

TTL (tiempo de vida): el dato se borra solo. **WRITETIME**: cuándo se escribió.

```
INSERT INTO alertas_por_dia (dia, hora, sensor, temperatura, mensaje) VALUES ('2026-10-16',
toTimestamp(now()), 'uthh-03', 36.1, 'Prueba con TTL') USING TTL 60;   -- vive 60 segundos
SELECT sensor, mensaje, TTL(mensaje), WRITETIME(mensaje) FROM alertas_por_dia WHERE dia =
'2026-10-16';   -- segundos que le quedan
```

Repite el SELECT después de un minuto: la alerta ya no está. **Contador:**

```
UPDATE conteo_por_dia SET lecturas = lecturas + 1 WHERE sensor = 'uthh-01' AND dia = '2026-
10-16';
UPDATE conteo_por_dia SET lecturas = lecturas + 1 WHERE sensor = 'uthh-01' AND dia = '2026-
10-16';
SELECT * FROM conteo_por_dia WHERE sensor = 'uthh-01';                -- lecturas = 2
```

BATCH: varias escrituras que se aplican juntas. Así guardará la aplicación cada lectura en varias tablas:

```
BEGIN BATCH
  INSERT INTO lecturas_por_sensor_dia (sensor, dia, hora, temperatura, humedad) VALUES
('uthh-03', '2026-10-16', '2026-10-16 15:30:00-0600', 36.2, 58);
  INSERT INTO ultima_lectura (sensor, hora, temperatura, humedad) VALUES ('uthh-03', '2026-
10-16 15:30:00-0600', 36.2, 58);
  INSERT INTO alertas_por_dia (dia, hora, sensor, temperatura, mensaje) VALUES ('2026-10-
16', '2026-10-16 15:30:00-0600', 'uthh-03', 36.2, 'Explanada supera 35');
APPLY BATCH;
SELECT * FROM alertas_por_dia WHERE dia = '2026-10-16';
```

Evidencia 3

Captura del upsert (una sola fila en `ultima_lectura`), del TTL antes y después del minuto, del contador y del BATCH.

Parte 5 · Construir la aplicación

11 Crear el proyecto de Node.js

Abre una **terminal nueva** (ícono **+** de la terminal) para no salir de cqlsh. En la carpeta de tu repositorio:

```
mkdir -p app/web                          # carpeta de la aplicacion y de sus paginas
cd app                                    # entra a la carpeta
npm init -y                               # crea package.json (la ficha del proyecto)
npm install cassandra-driver              # instala el controlador oficial de Cassandra
echo "node_modules/" > ../.gitignore      # no subir a GitHub las librerias descargadas
```

Vas a crear estos archivos dentro de `app`:

Archivo	Para qué sirve
`bd.js`	Conexión con Cassandra y la función que guarda una lectura en todas sus tablas.
`simulador.js`	Sensores simulados: una lectura por segundo, o la carga histórica de varios días.
`servidor.js`	Servidor web y API REST: recibe las peticiones de las páginas y ejecuta CQL.
`web/estilos.css`, `web/app.js`	Apariencia y funciones comunes de las páginas.
`web/index.html`, `sensor.html`, `registrar.html`	Tablero, gráfica por sensor y registro desde el celular.

12 La conexión y la escritura: bd.js

Crea `app/bd.js`. Es el corazón de la práctica: aquí está el CQL que usa toda la aplicación.

`app/bd.js`

```
// Estacion UTHH · conexion con Cassandra y escritura de lecturas.
// Lo usan el servidor web y el simulador.
const cassandra = require("cassandra-driver");        // controlador oficial de Cassandra para
Node.js
const { LocalDate } = cassandra.types;                // tipo para las columnas "date"

const KS = "estacion";                                // keyspace (el equivalente a la "base de
datos")
const ZONA = "America/Mexico_City";                   // zona horaria para decidir el "dia" de cada
lectura

// Cliente: se conecta al nodo de Cassandra que corre en Docker, puerto 9042
function crearCliente() {
  return new cassandra.Client({
    contactPoints: ["127.0.0.1"],                     // direccion del nodo
```

```
    localDataCenter: "datacenter1",                  // centro de datos (el nombre por defecto)
  });
}

// Dia local en formato AAAA-MM-DD (en Mexico, no en UTC)
function diaLocal(fecha = new Date()) {
  return new Intl.DateTimeFormat("en-CA", { timeZone: ZONA }).format(fecha);
}

// Arma el texto del CQL con los valores en lugar de los ?, para mostrarlo en la pagina
function mostrar(cql, params) {
  let i = 0;
  return cql.replace(/\?/g, () => {
    const v = params[i++];
    if (typeof v === "number") return String(v);
    if (v instanceof Date) return "'" + v.toISOString() + "'";
    return "'" + String(v) + "'";
  });
}

// Ejecuta una sentencia preparada (los ? se llenan con params) y la anota en la bitacora
async function ejecutar(cliente, cql, params, bitacora) {
  if (bitacora) bitacora.push(mostrar(cql, params));
  return cliente.execute(cql, params, { prepare: true });
}

// Lee la tabla sensores y la devuelve como { "uthh-01": {...}, ... }
async function leerSensores(cliente, bitacora) {
  const r = await ejecutar(cliente, `SELECT id, nombre, ubicacion, tipo, temp_max FROM
${KS}.sensores`, [], bitacora);
  const mapa = {};
  for (const f of r.rows) mapa[f.id] = { id: f.id, nombre: f.nombre, ubicacion: f.ubicacion, tipo:
f.tipo, temp_max: f.temp_max };
  return mapa;
}

// Guarda UNA lectura en todas las tablas que la necesitan (modelo desnormalizado):
//   lecturas_por_sensor_dia -> historial (grafica)        ultima_lectura -> tablero (se
sobrescribe)
//   alertas_por_dia         -> solo si supera el umbral    conteo_por_dia -> contador del dia
async function guardarLectura(cliente, sensor, lectura, bitacora, historico = false) {
  const { hora, temperatura, humedad } = lectura;
  const dia = diaLocal(hora);
  const fecha = LocalDate.fromString(dia);
  const lote = [{
    query: `INSERT INTO ${KS}.lecturas_por_sensor_dia (sensor, dia, hora, temperatura, humedad)
VALUES (?, ?, ?, ?, ?)`,
    params: [sensor.id, fecha, hora, temperatura, humedad],
  }];
  if (!historico) lote.push({                        // los datos viejos no deben pisar la ultima
lectura
    query: `INSERT INTO ${KS}.ultima_lectura (sensor, hora, temperatura, humedad) VALUES (?, ?, ?,
?)`,
    params: [sensor.id, hora, temperatura, humedad],
  });
  const alerta = temperatura > sensor.temp_max;
  if (alerta) lote.push({
    query: `INSERT INTO ${KS}.alertas_por_dia (dia, hora, sensor, temperatura, mensaje) VALUES (?,
?, ?, ?, ?)`,
    params: [fecha, hora, sensor.id, temperatura, sensor.nombre + " supera " + sensor.temp_max + "
°C"],
  });
  if (bitacora) bitacora.push("BEGIN BATCH", ...lote.map(l => "  " + mostrar(l.query, l.params) +
";"), "APPLY BATCH;");
  await cliente.batch(lote, { prepare: true });       // BATCH: se aplican juntas
```

```
  // Un contador no puede ir en el mismo BATCH que tablas normales
  await ejecutar(cliente, `UPDATE ${KS}.conteo_por_dia SET lecturas = lecturas + 1 WHERE sensor = ?
AND dia = ?`,
    [sensor.id, fecha], bitacora);
  return { dia, alerta };
}

module.exports = { KS, LocalDate, crearCliente, diaLocal, ejecutar, leerSensores, guardarLectura };
```

- `{ prepare: true }`: sentencias **preparadas**. Cassandra las analiza una vez y luego solo recibe los valores de los `?`; es más rápido y evita inyección de CQL.
- `guardarLectura()` es la **desnormalización** en código: una lectura se escribe en 3 o 4 tablas, cada una pensada para una pregunta.
- El contador va fuera del BATCH porque Cassandra no permite mezclar tablas de contadores con tablas normales.

13 Los sensores: simulador.js

Crea `app/simulador.js`:

app/simulador.js

```
// Estacion UTHH · simulador de sensores.
//   node simulador.js             -> una lectura por sensor cada segundo (Ctrl + C para detener)
//   node simulador.js historico 7  -> genera los ultimos 7 dias, una lectura cada 2 minutos
const { crearCliente, leerSensores, guardarLectura } = require("./bd");
const cliente = crearCliente();

// Temperatura y humedad "realistas": ciclo del dia (maximo a las 15 h) + ruido
function simular(sensor, fecha) {
  const hora = (fecha.getUTCHours() - 6 + 24) % 24 + fecha.getUTCMinutes() / 60;   // hora en
Mexico
  const exterior = sensor.tipo === "exterior";
  const base = exterior ? 28 : 25, amplitud = exterior ? 7 : 3;
  let t = base + amplitud * Math.cos(2 * Math.PI * (hora - 15) / 24) + (Math.random() - 0.5);
  if (Math.random() < 0.01) t += 5;                                                  // pico
ocasional
  const h = Math.max(30, Math.min(95, 75 - (t - base) * 2.5 + (Math.random() - 0.5) * 4));
  return { hora: fecha, temperatura: Math.round(t * 10) / 10, humedad: Math.round(h) };
}

async function enVivo(sensores) {
  console.log("Simulando " + sensores.length + " sensores cada segundo. Ctrl + C para detener.");
  setInterval(async () => {
    const ahora = new Date();
    const partes = [];
    for (const s of sensores) {
      const l = simular(s, ahora);
      const r = await guardarLectura(cliente, s, l);
      partes.push(s.id + " " + l.temperatura + "°C " + l.humedad + "%" + (r.alerta ? " ALERTA" :
""));
    }
    console.log(ahora.toLocaleTimeString("es-MX") + "   " + partes.join("  |  "));
  }, 1000);
}

async function historico(sensores, dias) {
  const paso = 2 * 60 * 1000, fin = Date.now() - 60000, inicio = fin - dias * 86400000;
  const tareas = [];
  for (const s of sensores) for (let t = inicio; t <= fin; t += paso) tareas.push([s, simular(s,
```

```
new Date(t))]);
  console.log("Escribiendo " + tareas.length + " lecturas de " + dias + " dias...");
  const t0 = Date.now();
  let i = 0;
  async function trabajador() {                        // 50 trabajadores escriben al mismo tiempo
    while (i < tareas.length) {
      const [s, l] = tareas[i++];
      await guardarLectura(cliente, s, l, null, true);
      if (i % 2000 === 0) console.log("  " + i + " lecturas...");
    }
  }
  await Promise.all(Array.from({ length: 50 }, trabajador));
  const seg = (Date.now() - t0) / 1000;
  console.log("Listo: " + tareas.length + " lecturas en " + seg.toFixed(1) + " s (" +
Math.round(tareas.length / seg) + " por segundo).");
  await cliente.shutdown();
}

leerSensores(cliente).then(mapa => {
  const sensores = Object.values(mapa);
  if (!sensores.length) { console.log("Primero inserta los sensores en estacion.sensores.");
process.exit(1); }
  if (process.argv[2] === "historico") return historico(sensores, Number(process.argv[3]) || 7);
  return enVivo(sensores);
}).catch(e => { console.error("Error:", e.message); process.exit(1); });
```

Pruébalo (desde la carpeta `app`). Primero la **carga histórica**, que mide cuántas escrituras por segundo soporta tu Cassandra:

```
node simulador.js historico 7            # 7 dias de lecturas, una cada 2 minutos por
sensor (~15,000)
```

Después, en otra terminal (también en `app`), los sensores **en vivo**. Déjalo corriendo:

```
cd app                                   # si la terminal es nueva
node simulador.js                        # una lectura por sensor cada segundo; Ctrl + C
para detener
```

Compruébalo en cqlsh

`SELECT * FROM ultima_lectura;` cambia cada segundo, y `SELECT * FROM conteo_por_dia WHERE sensor = 'uthh-03';` muestra cuántas lecturas lleva cada día.

Evidencia 4

Captura del resultado de la carga histórica (lecturas por segundo) y del simulador en vivo.

14 La API: servidor.js

Crea `app/servidor.js`. Usa el servidor HTTP que ya trae Node.js; no necesita librerías extra.

app/servidor.js

```
// Estacion UTHH · servidor web y API REST sobre Cassandra.
// Entrega las paginas de la carpeta web/ y responde las rutas /api/... con JSON.
// Cada respuesta trae "cql": las sentencias que se ejecutaron en Cassandra.
```

```
const http = require("http");                          // servidor web incluido en Node.js
const fs = require("fs");                              // para leer los archivos de web/
const path = require("path");
const { KS, LocalDate, crearCliente, diaLocal, ejecutar, guardarLectura, leerSensores } =
require("./bd");

const PUERTO = 3000;
const WEB = path.join(__dirname, "web");
const TABLAS = ["sensores", "lecturas_por_sensor_dia", "ultima_lectura", "alertas_por_dia",
"conteo_por_dia"];
const cliente = crearCliente();                        // UNA conexion para todo el servidor

// ------------------------------------------------------------------ rutas de la API
// Cada ruta recibe la URL ya separada y devuelve un objeto que se enviara como JSON.

// GET /api/estado : version de Cassandra y tablas que ya existen en el keyspace
async function estado() {
  const cql = [];
  const v = (await ejecutar(cliente, "SELECT release_version FROM system.local", [], cql)).first();
  const t = await ejecutar(cliente, "SELECT table_name FROM system_schema.tables WHERE
keyspace_name = ?", [KS], cql);
  const tablas = t.rows.map(r => r.table_name);
  return { version: v.release_version, tablas, faltan: TABLAS.filter(x => !tablas.includes(x)), cql
};
}

// GET /api/sensores : catalogo + ultima lectura + lecturas de hoy de cada sensor
async function sensores() {
  const cql = [];
  const hoy = diaLocal();
  const catalogo = await leerSensores(cliente, cql);
  const ultimas = {};
  const u = await ejecutar(cliente, `SELECT sensor, hora, temperatura, humedad FROM
${KS}.ultima_lectura`, [], cql);
  for (const r of u.rows) ultimas[r.sensor] = { hora: r.hora, temperatura: r.temperatura, humedad:
r.humedad };
  const lista = [];
  for (const s of Object.values(catalogo)) {
    const c = await ejecutar(cliente, `SELECT lecturas FROM ${KS}.conteo_por_dia WHERE sensor = ?
AND dia = ?`,
      [s.id, LocalDate.fromString(hoy)], cql);
    const fila = c.first();
    lista.push({ ...s, ultima: ultimas[s.id] || null, lecturasHoy: fila ? Number(fila.lecturas) : 0
});
  }
  lista.sort((a, b) => a.id.localeCompare(b.id));
  return { hoy, sensores: lista, cql };
}

// GET /api/sensores/uthh-01/lecturas?dia=AAAA-MM-DD&limite=1000 : UNA particion (sensor, dia)
async function lecturas(id, q) {
  const cql = [];
  const dia = q.get("dia") || diaLocal();
  const limite = Math.min(Number(q.get("limite")) || 1000, 5000);
  const r = await ejecutar(cliente,
    `SELECT hora, temperatura, humedad FROM ${KS}.lecturas_por_sensor_dia WHERE sensor = ? AND dia
= ? LIMIT ?`,
    [id, LocalDate.fromString(dia), limite], cql);
  return { sensor: id, dia, lecturas: r.rows.map(x => ({ hora: x.hora, temperatura: x.temperatura,
humedad: x.humedad })), cql };
}

// GET /api/alertas?dia=AAAA-MM-DD : las alertas de un dia (una sola particion)
async function alertas(q) {
  const cql = [];
```

```
  const dia = q.get("dia") || diaLocal();
  const r = await ejecutar(cliente,
    `SELECT hora, sensor, temperatura, mensaje FROM ${KS}.alertas_por_dia WHERE dia = ? LIMIT 200`,
    [LocalDate.fromString(dia)], cql);
  return { dia, alertas: r.rows.map(x => ({ hora: x.hora, sensor: x.sensor, temperatura:
x.temperatura, mensaje: x.mensaje })), cql };
}

// POST /api/lecturas  {"sensor":"uthh-01","temperatura":27.5,"humedad":70}
async function registrar(datos) {
  const cql = [];
  const s = (await leerSensores(cliente, cql))[datos.sensor];
  if (!s) throw new Error("No existe el sensor " + datos.sensor);
  const t = Number(datos.temperatura), h = Number(datos.humedad);
  if (!(t >= -20 && t <= 60)) throw new Error("La temperatura debe estar entre -20 y 60 °C");
  if (!(h >= 0 && h <= 100)) throw new Error("La humedad debe estar entre 0 y 100 %");
  const r = await guardarLectura(cliente, s, { hora: new Date(), temperatura: t, humedad: h },
cql);
  return { sensor: s.id, dia: r.dia, alerta: r.alerta, cql };
}

// ------------------------------------------------------------------ servidor
const TIPOS = { ".html": "text/html; charset=utf-8", ".css": "text/css; charset=utf-8", ".js":
"text/javascript; charset=utf-8" };

http.createServer(async (req, res) => {
  const url = new URL(req.url, "http://localhost");
  const ruta = url.pathname;
  // 1) Archivos de la carpeta web/ (index.html, estilos.css, app.js...)
  if (!ruta.startsWith("/api/")) {
    const archivo = path.join(WEB, ruta === "/" ? "index.html" : path.normalize(ruta));
    if (!archivo.startsWith(WEB)) { res.writeHead(403); return res.end(); }
    return fs.readFile(archivo, (e, contenido) => {
      if (e) { res.writeHead(404); return res.end("No encontrado"); }
      res.writeHead(200, { "Content-Type": TIPOS[path.extname(archivo)] || "text/plain" });
      res.end(contenido);
    });
  }
  // 2) Rutas de la API: siempre responden JSON
  let codigo = 200, respuesta;
  try {
    let m;
    if (req.method === "GET" && ruta === "/api/estado") respuesta = await estado();
    else if (req.method === "GET" && ruta === "/api/sensores") respuesta = await sensores();
    else if (req.method === "GET" && (m = ruta.match(/^\/api\/sensores\/([\w-]+)\/lecturas$/)))
respuesta = await lecturas(m[1], url.searchParams);
    else if (req.method === "GET" && ruta === "/api/alertas") respuesta = await
alertas(url.searchParams);
    else if (req.method === "POST" && ruta === "/api/lecturas") {
      let cuerpo = "";
      for await (const parte of req) cuerpo += parte;   // lee el JSON que envio la pagina
      respuesta = await registrar(JSON.parse(cuerpo || "{}"));
    }
    else { codigo = 404; respuesta = { error: "Ruta no encontrada: " + ruta }; }
  } catch (e) {
    codigo = 400;
    respuesta = { error: e.message };
    console.error("Error:", e.message);
  }
  res.writeHead(codigo, { "Content-Type": "application/json; charset=utf-8" });
  res.end(JSON.stringify(respuesta));
}).listen(PUERTO, () => console.log("Estacion UTHH en el puerto " + PUERTO + " (pestana Puertos ->
3000)"));
```

Ruta	Tabla que consulta
`GET /api/estado`	`system.local` y `system_schema.tables` (metadatos)
`GET /api/sensores`	`sensores`, `ultima_lectura`, `conteo_por_dia`
`GET /api/sensores/:id/lecturas?dia=&limite=`	`lecturas_por_sensor_dia` (una partición)
`GET /api/alertas?dia=`	`alertas_por_dia` (una partición)
`POST /api/lecturas`	Escribe en 3 o 4 tablas con `guardarLectura()`

15 Las páginas: carpeta web

Crea los cinco archivos en `app/web`. Los estilos no son parte de la base de datos: cópialos tal cual.

app/web/estilos.css

```
/* Estacion UTHH · estilos (no son parte de la base de datos: copialos tal cual) */
:root { --fondo:#F3F6F9; --papel:#FFFFFF; --tinta:#12212E; --suave:#556574; --linea:#D6E0E8;
        --azul:#1B5E8C; --azul2:#E4F0F8; --naranja:#B5560F; --naranja2:#FCEBDD; --rojo:#B3261E; --
rojo2:#FCE9E7; }
@media (prefers-color-scheme: dark) {
  :root { --fondo:#0C141B; --papel:#142029; --tinta:#E3EDF5; --suave:#9AAEBF; --linea:#263846;
          --azul:#6CB8EC; --azul2:#16293A; --naranja:#F2A55E; --naranja2:#3A2814; --rojo:#FF8A80; -
-rojo2:#3A1D1B; color-scheme: dark; }
}
* { box-sizing: border-box; }
[hidden] { display: none !important; }
body { margin: 0; background: var(--fondo); color: var(--tinta); font: 15px/1.5 system-ui, "Segoe
UI", Roboto, sans-serif; }
header { background: #1B5E8C; color: #fff; }
header nav { max-width: 1100px; margin: 0 auto; padding: 10px 16px; display: flex; gap: 6px; flex-
wrap: wrap; align-items: center; }
header a { color: #fff; text-decoration: none; padding: 6px 10px; border-radius: 8px; font-weight:
600; }
header a[aria-current], header a:hover { background: rgba(255,255,255,.18); }
header b { margin-right: 12px; }
#estado { margin-left: auto; font-size: .85rem; }
main { max-width: 1100px; margin: 0 auto; padding: 20px 16px 48px; display: grid; gap: 16px; }
h1, h2, h3 { margin: 0; line-height: 1.2; }
p { margin: 0; }
.nota { color: var(--suave); font-size: .9rem; }
.tarjeta { background: var(--papel); border: 1px solid var(--linea); border-radius: 12px; padding:
16px; display: grid; gap: 10px; min-width: 0; }
.rejilla { display: grid; grid-template-columns: repeat(auto-fit, minmax(260px, 1fr)); gap: 16px; }
.sensor .temp { font-size: 2.8rem; font-weight: 800; line-height: 1; }
.sensor.alerta { border-color: var(--naranja); box-shadow: inset 4px 0 0 var(--naranja); }
.chip { display: inline-block; font-size: .75rem; font-weight: 700; padding: 2px 8px; border-
radius: 999px; background: var(--azul2); color: var(--azul); }
.chip.alerta { background: var(--naranja2); color: var(--naranja); }
.aviso { border-radius: 8px; padding: 10px 12px; border-left: 4px solid var(--azul); background:
var(--azul2); }
.aviso.error { border-color: var(--rojo); background: var(--rojo2); }
label { display: grid; gap: 4px; font-size: .85rem; font-weight: 600; color: var(--suave); }
input, select, button { font: inherit; padding: 8px 10px; border-radius: 8px; border: 1px solid
var(--linea); background: var(--fondo); color: var(--tinta); }
button { background: var(--azul); color: #fff; border: 0; font-weight: 700; cursor: pointer; }
```

```
.campos { display: grid; grid-template-columns: repeat(auto-fit, minmax(150px, 1fr)); gap: 10px; }
.tabla { overflow-x: auto; }
table { width: 100%; border-collapse: collapse; font-size: .9rem; }
th, td { text-align: left; padding: 7px 8px; border-bottom: 1px solid var(--linea); }
th { font-size: .75rem; text-transform: uppercase; color: var(--suave); }
pre { background: #0E1B26; color: #D8EAF7; padding: 10px 12px; border-radius: 8px; font-size:
.8rem; overflow-x: auto; margin: 0; }
summary { cursor: pointer; font-weight: 600; color: var(--azul); }
svg text { font: 12px system-ui, sans-serif; fill: var(--suave); }
```

app/web/app.js

```
// Estacion UTHH · funciones que comparten todas las paginas.
// Las paginas NO hablan con Cassandra: le piden los datos a la API (/api/...)
// y la API responde JSON con los datos y con el CQL que ejecuto.

// Pide algo a la API y devuelve el JSON; si hay error, lanza el mensaje del servidor
async function api(ruta, metodo = "GET", datos) {
  const opciones = { method: metodo, headers: { "Content-Type": "application/json" } };
  if (datos) opciones.body = JSON.stringify(datos);
  let r;
  try { r = await fetch("/api" + ruta, opciones); }
  catch (e) { throw new Error("Sin conexion con el servidor: revisa que node servidor.js siga
corriendo."); }
  const json = await r.json().catch(() => ({}));
  if (!r.ok) throw new Error(json.error || "Error HTTP " + r.status);
  return json;
}

// Crea un elemento HTML con clase y texto
function el(etiqueta, clase, texto) {
  const e = document.createElement(etiqueta);
  if (clase) e.className = clase;
  if (texto !== undefined) e.textContent = texto;
  return e;
}

// Formatos: Cassandra guarda la hora en UTC; el navegador la muestra en hora local
const horaLocal = s => new Date(s).toLocaleTimeString("es-MX");
const num = n => (n === null || n === undefined) ? "—" : Number(n).toFixed(1);
function hoyLocal() {
  const d = new Date();
  return d.getFullYear() + "-" + String(d.getMonth() + 1).padStart(2, "0") + "-" +
String(d.getDate()).padStart(2, "0");
}
function hace(s) {
  const seg = Math.round((Date.now() - new Date(s)) / 1000);
  return seg < 60 ? "hace " + Math.max(seg, 0) + " s" : seg < 3600 ? "hace " + Math.round(seg / 60)
+ " min" : new Date(s).toLocaleString("es-MX");
}

// Muestra un aviso (tipo "error" en rojo) dentro de un contenedor
function aviso(contenedor, texto, tipo = "") {
  contenedor.textContent = "";
  if (texto) contenedor.append(el("div", "aviso " + tipo, texto));
}

// Muestra el CQL que ejecuto la API
function verCql(contenedor, lineas) {
  contenedor.textContent = "";
  const d = el("details");
  d.append(el("summary", null, "CQL que ejecuto la API (" + lineas.length + ")"), el("pre", null,
lineas.join("\n")));
  contenedor.append(d);
```

```
}

// Menu superior y estado de la conexion con Cassandra
function menu(actual) {
  const nav = el("nav");
  nav.append(el("b", null, "Estacion UTHH"));
  for (const [pagina, texto] of [["index.html", "Tablero"], ["sensor.html", "Sensor"],
["registrar.html", "Registrar"]]) {
    const a = el("a", null, texto); a.href = pagina;
    if (pagina === actual) a.setAttribute("aria-current", "page");
    nav.append(a);
  }
  const estado = el("span", null, "Conectando..."); estado.id = "estado";
  nav.append(estado);
  const header = el("header"); header.append(nav);
  document.body.prepend(header);
  api("/estado")
    .then(e => { estado.textContent = "● Cassandra " + e.version + (e.faltan.length ? " · faltan
tablas" : ""); })
    .catch(e => { estado.textContent = "● Sin conexion"; estado.title = e.message; });
}

// Grafica SVG: temperatura (naranja, eje izquierdo) y humedad (azul, 0 a 100 %)
function grafica(contenedor, lecturas) {
  contenedor.textContent = "";
  if (!lecturas.length) { contenedor.append(el("p", "nota", "Sin lecturas en esta particion (sensor
+ dia).")); return; }
  const d = lecturas.slice().reverse();                       // vienen de la mas nueva a la mas
vieja
  const W = 800, H = 280, m = 40;
  const t0 = new Date(d[0].hora).getTime(), t1 = Math.max(new Date(d[d.length - 1].hora).getTime(),
t0 + 1);
  const temps = d.map(x => x.temperatura);
  const tMin = Math.floor(Math.min(...temps) - 1), tMax = Math.ceil(Math.max(...temps) + 1);
  const x = t => m + (t - t0) / (t1 - t0) * (W - 2 * m);
  const yT = v => 10 + (1 - (v - tMin) / (tMax - tMin)) * (H - m - 10);
  const yH = v => 10 + (1 - v / 100) * (H - m - 10);
  const linea = (f, color) => `<path fill="none" stroke="${color}" stroke-width="2" d="${d.map((p,
i) => (i ? "L" : "M") + x(new Date(p.hora).getTime()).toFixed(1) + " " + f(p).toFixed(1)).join("
")}"/>`;
  let ejes = "";
  for (let k = 0; k <= 4; k++) {
    const v = tMin + (tMax - tMin) * k / 4, t = t0 + (t1 - t0) * k / 4;
    ejes += `<line x1="${m}" x2="${W - m}" y1="${yT(v)}" y2="${yT(v)}" stroke="#8899AA" stroke-
opacity=".25"/>`
      + `<text x="${m - 6}" y="${yT(v) + 4}" text-anchor="end">${v.toFixed(0)}°</text>`
      + `<text x="${W - m + 6}" y="${yH(k * 25) + 4}">${k * 25}%</text>`
      + `<text x="${x(t)}" y="${H - 12}" text-anchor="middle">${new Date(t).toLocaleTimeString("es-
MX", { hour: "2-digit", minute: "2-digit" })}</text>`;
  }
  contenedor.innerHTML = `<svg viewBox="0 0 ${W} ${H}" width="100%" role="img" aria-label="Grafica
de temperatura y humedad">`
    + ejes + linea(p => yH(p.humedad), "#3A9BD9") + linea(p => yT(p.temperatura), "#D9822B") + "
</svg>";
}
```

app/web/index.html

```
<!doctype html>
<html lang="es">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Tablero · Estacion UTHH</title>
```

```
<link rel="stylesheet" href="estilos.css">
</head>
<body>
<main>
  <h1>Estacion meteorologica UTHH</h1>
  <p class="nota">Se actualiza cada 2 segundos leyendo las tablas <b>sensores</b>,
<b>ultima_lectura</b>, <b>conteo_por_dia</b> y <b>alertas_por_dia</b>.</p>
  <div id="aviso"></div>
  <section class="rejilla" id="sensores"></section>
  <div class="rejilla">
    <section class="tarjeta">
      <h2>Alertas de hoy</h2>
      <div class="tabla"><table>
        <thead><tr><th>Hora</th><th>Sensor</th><th>Temp.</th></tr></thead>
        <tbody id="alertas"></tbody>
      </table></div>
    </section>
    <section class="tarjeta" style="justify-items:center;text-align:center">
      <h2>Registra una lectura desde tu celular</h2>
      <div id="qr" style="background:#fff;padding:8px;border-radius:8px;line-height:0"></div>
      <p class="nota">El puerto 3000 debe estar en Public.</p>
    </section>
  </div>
  <div id="cql"></div>
</main>
<script src="https://cdnjs.cloudflare.com/ajax/libs/qrcode-generator/1.4.4/qrcode.min.js"></script>
<script src="app.js"></script>
<script>
menu("index.html");

async function cargar() {
  try {
    const r = await api("/sensores");                 // GET /api/sensores
    const a = await api("/alertas?dia=" + r.hoy);     // GET /api/alertas?dia=AAAA-MM-DD
    // Una tarjeta por sensor
    const cont = document.getElementById("sensores");
    cont.textContent = "";
    if (!r.sensores.length) aviso(cont, "No hay sensores: insertalos en la tabla
estacion.sensores.");
    for (const s of r.sensores) {
      const alto = s.ultima && s.ultima.temperatura > s.temp_max;
      const t = el("a", "tarjeta sensor" + (alto ? " alerta" : ""));
      t.href = "sensor.html?id=" + s.id;
      t.style.color = "inherit"; t.style.textDecoration = "none";
      t.append(el("h3", null, s.nombre), el("p", "nota", s.ubicacion + " · " + s.id),
        el("div", "temp", s.ultima ? num(s.ultima.temperatura) + " °C" : "—"),
        el("p", null, "Humedad " + (s.ultima ? s.ultima.humedad + " %" : "—") + " · umbral " +
s.temp_max + " °C"),
        el("p", "nota", (s.ultima ? "Ultima lectura " + hace(s.ultima.hora) : "Sin lecturas") + " ·
" + s.lecturasHoy + " lecturas hoy"));
      if (alto) t.append(el("span", "chip alerta", "Sobre el umbral"));
      cont.append(t);
    }
    // Alertas del dia (las 10 mas recientes)
    const tb = document.getElementById("alertas");
    tb.textContent = "";
    for (const x of a.alertas.slice(0, 10)) {
      const tr = el("tr");
      tr.append(el("td", null, horaLocal(x.hora)), el("td", null, x.sensor), el("td", null,
num(x.temperatura) + " °C"));
      tb.append(tr);
    }
    if (!a.alertas.length) tb.innerHTML = '<tr><td colspan="3" class="nota">Sin alertas hoy</td>
</tr>';
    verCql(document.getElementById("cql"), r.cql.concat(a.cql));
```

```
    aviso(document.getElementById("aviso"), "");
  } catch (e) {
    aviso(document.getElementById("aviso"), e.message, "error");
  }
}

// Codigo QR que lleva a la pagina para registrar lecturas
try {
  const q = qrcode(0, "M");
  q.addData(location.origin + "/registrar.html");
  q.make();
  document.getElementById("qr").innerHTML = q.createSvgTag(5, 2);
} catch (e) { /* sin internet no se dibuja el QR */ }

cargar();
setInterval(cargar, 2000);
</script>
</body>
</html>
```

app/web/sensor.html

```
<!doctype html>
<html lang="es">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Sensor · Estacion UTHH</title>
<link rel="stylesheet" href="estilos.css">
</head>
<body>
<main>
  <h1>Lecturas de un sensor</h1>
  <p class="nota">Lee <b>una sola particion</b> de <b>lecturas_por_sensor_dia</b>: la del sensor y
el dia elegidos.</p>
  <form class="tarjeta campos" id="filtro">
    <label>Sensor <select id="sensor"></select></label>
    <label>Dia <input id="dia" type="date"></label>
    <label>LIMIT <select id="limite"><option>100</option><option selected>1000</option>
<option>5000</option></select></label>
  </form>
  <div id="aviso"></div>
  <section class="tarjeta">
    <h2 id="resumen">Temperatura (naranja) y humedad (azul)</h2>
    <div id="grafica"></div>
  </section>
  <section class="tarjeta">
    <h2>Ultimas 10 lecturas</h2>
    <div class="tabla"><table>
      <thead><tr><th>Hora</th><th>Temperatura</th><th>Humedad</th></tr></thead>
      <tbody id="tabla"></tbody>
    </table></div>
  </section>
  <div id="cql"></div>
</main>
<script src="app.js"></script>
<script>
menu("sensor.html");
const parametros = new URLSearchParams(location.search);
const $sensor = document.getElementById("sensor"), $dia = document.getElementById("dia"), $limite =
document.getElementById("limite");
$dia.value = hoyLocal();

async function iniciar() {
```

```
  const r = await api("/sensores");
  for (const s of r.sensores) {
    const o = el("option", null, s.nombre + " (" + s.id + ")");
    o.value = s.id;
    $sensor.append(o);
  }
  if (parametros.get("id")) $sensor.value = parametros.get("id");
  cargar();
}

async function cargar() {
  if (!$sensor.value) return;
  try {
    // GET /api/sensores/uthh-01/lecturas?dia=AAAA-MM-DD&limite=1000
    const r = await api("/sensores/" + $sensor.value + "/lecturas?dia=" + $dia.value + "&limite=" +
$limite.value);
    grafica(document.getElementById("grafica"), r.lecturas);
    const temps = r.lecturas.map(l => l.temperatura);
    document.getElementById("resumen").textContent = r.lecturas.length + " lecturas" +
      (temps.length ? " · minima " + num(Math.min(...temps)) + " °C · maxima " +
num(Math.max(...temps)) + " °C" : "");
    const tb = document.getElementById("tabla");
    tb.textContent = "";
    for (const l of r.lecturas.slice(0, 10)) {
      const tr = el("tr");
      tr.append(el("td", null, horaLocal(l.hora)), el("td", null, num(l.temperatura) + " °C"),
el("td", null, l.humedad + " %"));
      tb.append(tr);
    }
    verCql(document.getElementById("cql"), r.cql);
    aviso(document.getElementById("aviso"), "");
  } catch (e) {
    aviso(document.getElementById("aviso"), e.message, "error");
  }
}

document.getElementById("filtro").addEventListener("change", cargar);
setInterval(() => { if ($dia.value === hoyLocal()) cargar(); }, 5000);   // si es hoy, se actualiza
sola
iniciar().catch(e => aviso(document.getElementById("aviso"), e.message, "error"));
</script>
</body>
</html>
```

app/web/registrar.html

```
<!doctype html>
<html lang="es">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Registrar · Estacion UTHH</title>
<link rel="stylesheet" href="estilos.css">
</head>
<body>
<main style="max-width:640px">
  <h1>Registrar una lectura</h1>
  <p class="nota">Eres un sensor manual: envia una lectura y mira en cuantas tablas se guarda.</p>
  <form class="tarjeta" id="form">
    <label>Sensor <select id="sensor"></select></label>
    <div class="campos">
      <label>Temperatura (°C) <input id="temperatura" type="number" step="0.1" value="27.5"
inputmode="decimal"></label>
      <label>Humedad (%) <input id="humedad" type="number" step="1" value="70" inputmode="numeric">
```

```
</label>
    </div>
    <button type="submit">Guardar lectura</button>
  </form>
  <div id="aviso"></div>
  <section class="tarjeta" id="resultado" hidden>
    <h2>Asi se guardo en Cassandra</h2>
    <p class="nota">Una lectura se escribe en varias tablas, cada una pensada para una consulta.
Las inserciones van juntas en un BATCH; el contador va aparte.</p>
    <pre id="cql"></pre>
  </section>
</main>
<script src="app.js"></script>
<script>
menu("registrar.html");

api("/sensores").then(r => {
  for (const s of r.sensores) {
    const o = el("option", null, s.nombre + " · umbral " + s.temp_max + " °C");
    o.value = s.id;
    document.getElementById("sensor").append(o);
  }
}).catch(e => aviso(document.getElementById("aviso"), e.message, "error"));

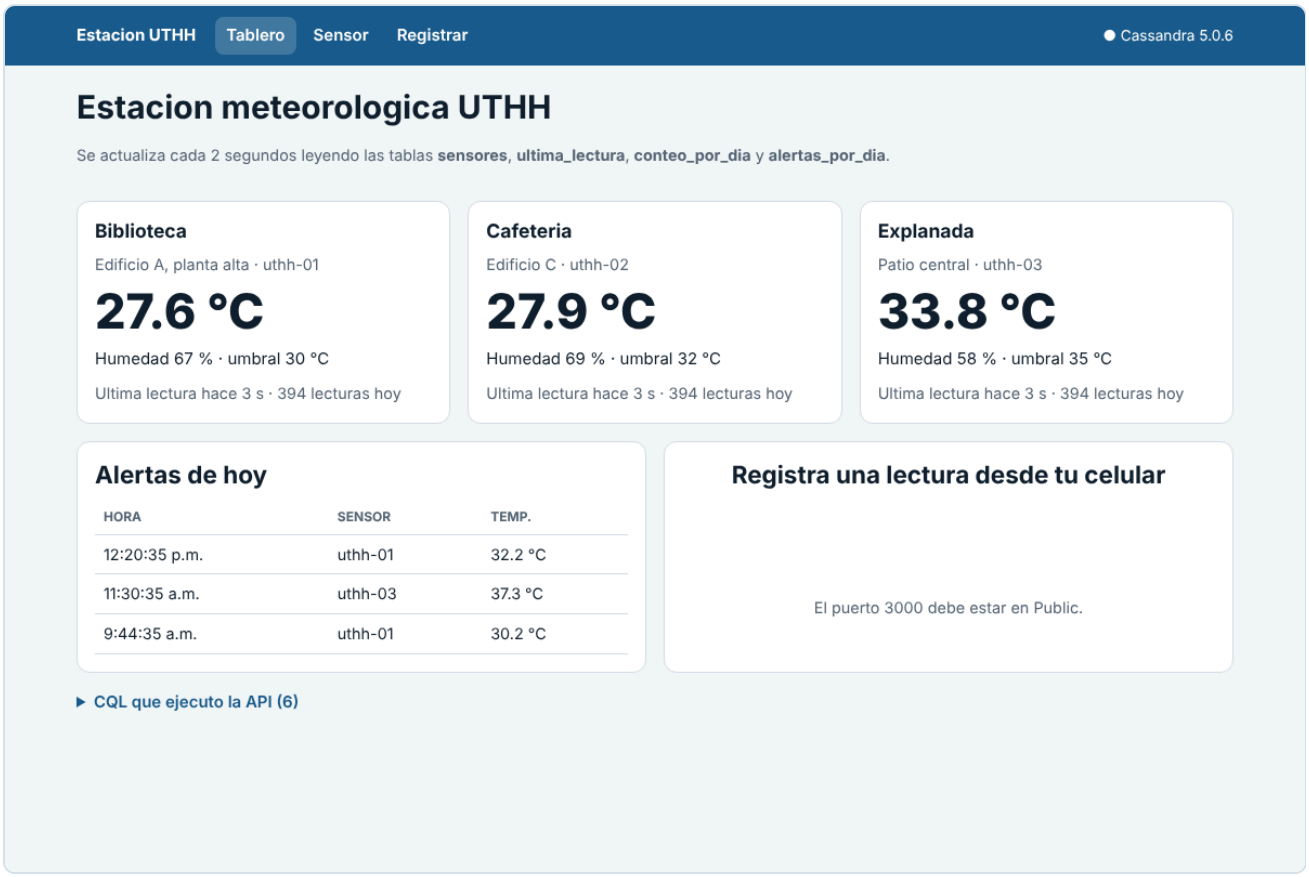
document.getElementById("form").addEventListener("submit", async evento => {
  evento.preventDefault();                    // no recargar la pagina
  try {
    // POST /api/lecturas con la lectura en JSON
    const r = await api("/lecturas", "POST", {
      sensor: document.getElementById("sensor").value,
      temperatura: Number(document.getElementById("temperatura").value),
      humedad: Number(document.getElementById("humedad").value),
    });
    aviso(document.getElementById("aviso"), r.alerta
      ? "Guardada. Supera el umbral: tambien se registro una ALERTA."
      : "Guardada en la particion (" + r.sensor + ", " + r.dia + ").");
    document.getElementById("cql").textContent = r.cql.join("\n");
    document.getElementById("resultado").hidden = false;
  } catch (e) {
    aviso(document.getElementById("aviso"), e.message, "error");
  }
});
</script>
</body>
</html>
```

16 Encender y abrir la aplicación

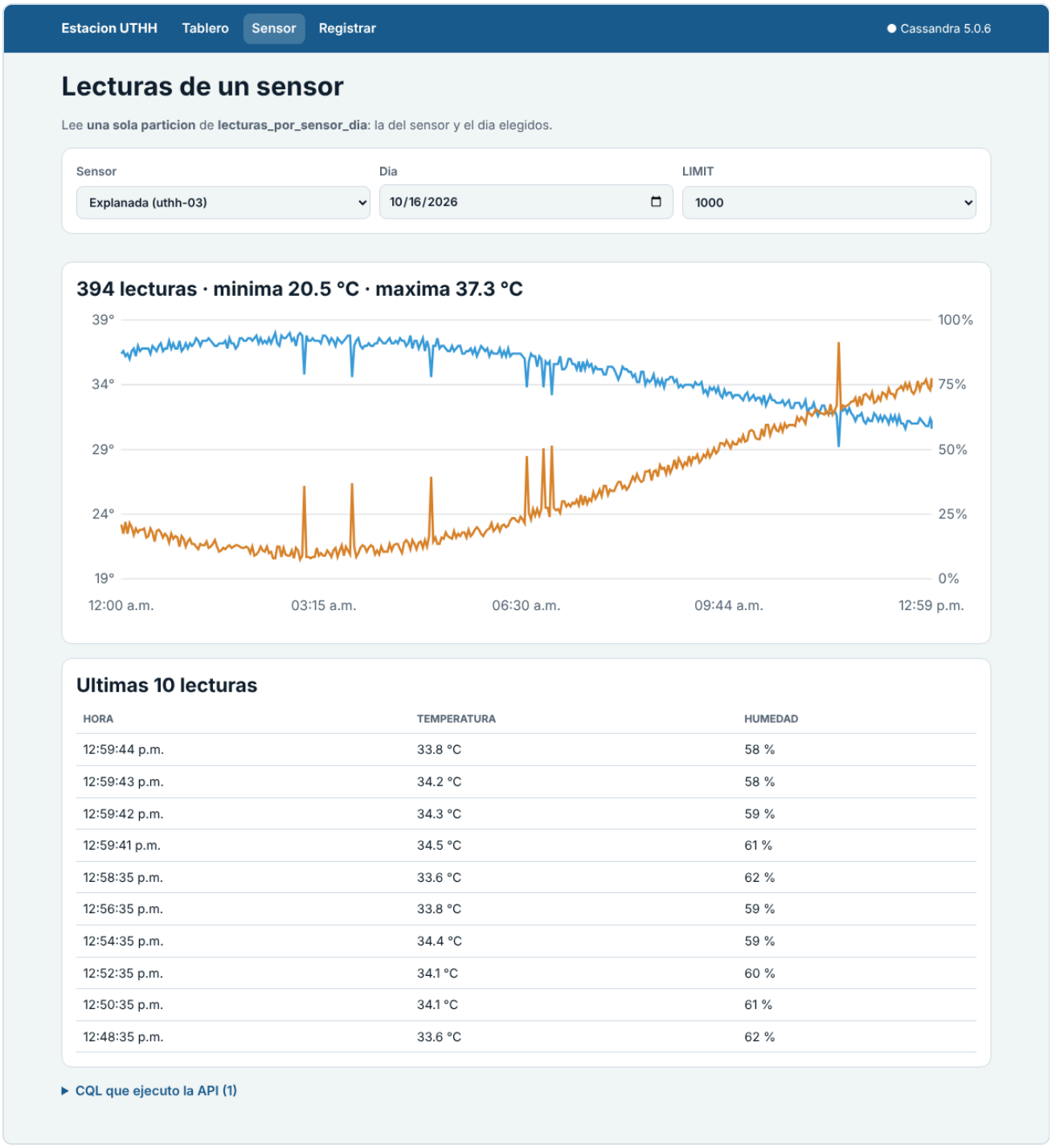
En una tercera terminal (deja el simulador corriendo en la otra):

```
cd app
node servidor.js                        # → Estacion UTHH en el puerto 3000
```

1. Pestaña **Puertos**: aparece el **3000**. Clic en el globo 🌐 para abrir el tablero.
2. Para el celular: clic derecho en el 3000 → **Visibilidad del puerto → Public** y escanea el QR del tablero. Si aparece el aviso de GitHub *"You are about to access a development port"*, presiona **Continue**.
3. **Nunca** hagas público el puerto 9042 (el de Cassandra): solo la API debe quedar expuesta.

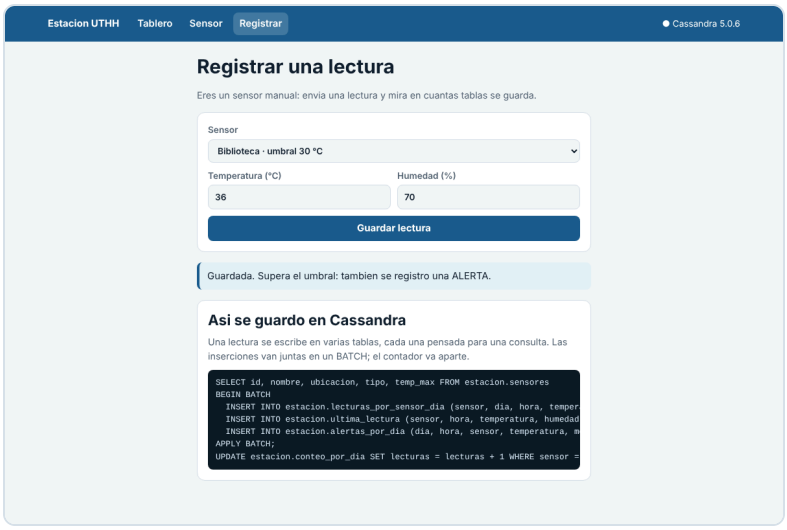


Tablero: la temperatura de cada sensor cambia cada segundo; la tarjeta con borde naranja supera su umbral. Abajo, el CQL que ejecutó la API.



Sensor: la gráfica es UNA partición (sensor + día). Con el histórico se ve el ciclo del día: más calor a las 15 h.





En el celular y en Registrar: una lectura manual se guarda con un BATCH en varias tablas y la página muestra exactamente el CQL.

El puerto vuelve solo a Private

Cada vez que el codespace se reinicia, el puerto 3000 regresa a Private: en el celular verás el inicio de sesión de GitHub o un error 404. Vuelve a ponerlo en Public.

Evidencia 5

Capturas del tablero, de la gráfica de un sensor con el histórico y de una lectura registrada desde el celular que haya generado una alerta (temperatura mayor que el umbral).

Parte 6 · Observar cómo trabaja Cassandra

17 Particiones, volumen y rendimiento

En `cqlsh`:

```
SELECT COUNT(*) FROM lecturas_por_sensor_dia WHERE sensor = 'uthh-03' AND dia = '2026-10-16';   -- filas de UNA particion (usa la fecha de hoy)
SELECT DISTINCT sensor, dia FROM lecturas_por_sensor_dia;
-- todas las particiones que existen
SELECT dia, lecturas FROM conteo_por_dia WHERE sensor = 'uthh-01';
-- lecturas por dia de un sensor
```

En la terminal normal, las estadísticas del nodo:

```
docker exec cassandra nodetool flush estacion                    # escribe a disco lo que esta en memoria
docker exec cassandra nodetool tablestats estacion.lecturas_por_sensor_dia | head -30   # escrituras, tamano, particiones
```

- En `tablestats` busca *Local write count* (escrituras), *Space used* (tamaño en disco) y *Number of partitions (estimate)*.
- Cassandra escribe primero en memoria (memtable) y en una bitácora (commit log); `flush` lo pasa a archivos en disco (SSTables). Por eso escribe tan rápido.

> **Evidencia 6**
> Captura del COUNT de una partición, de las particiones con DISTINCT y de `tablestats`.

Parte 7 · Laboratorio: lo que Cassandra no te deja hacer

18 Consultas prohibidas y por qué

En `cqlsh`. Cada sentencia **debe dar error**; lee el mensaje:

```
SELECT * FROM lecturas_por_sensor_dia WHERE temperatura > 35;          -- 1. filtrar por una columna que no es llave
SELECT * FROM lecturas_por_sensor_dia WHERE sensor = 'uthh-01';        -- 2. llave de particion incompleta (falta dia)
SELECT * FROM lecturas_por_sensor_dia WHERE sensor = 'uthh-01' AND dia = '2026-10-16' ORDER BY temperatura;   -- 3. ordenar por algo que no es agrupamiento
INSERT INTO conteo_por_dia (sensor, dia, lecturas) VALUES ('uthh-01', '2026-10-16', 5);   -- 4. un contador no se inserta
SELECT * FROM lecturas l JOIN sensores s ON l.sensor = s.id;           -- 5. no existe JOIN
INSERT INTO sensores (id, temp_max) VALUES ('uthh-09', 'muy alta');    -- 6. tipo de dato incorrecto
```

#	Qué dice el error (resumen)	Por qué
1	Pide `ALLOW FILTERING`	Tendría que leer TODAS las particiones de todos los nodos. Cassandra no lo hace sin que lo pidas.
2	Faltan columnas de la llave de partición	Sin `dia` no sabe en qué partición (ni en qué nodo) buscar.
3	ORDER BY solo con columnas de agrupamiento	El orden se decide al diseñar la tabla, no al consultar.
4	Los contadores solo se modifican con UPDATE	Un contador es una suma distribuida; no se le asigna un valor.
5	Error de sintaxis	CQL no tiene JOIN: los datos que se consultan juntos se guardan juntos.
6	Tipo inválido para `temp_max`	Aunque el esquema es por consulta, cada columna sí tiene tipo.

Prueba la 1 con `ALLOW FILTERING` al final: funciona aquí porque hay pocos datos y un solo nodo, pero en producción con millones de filas sería muy lenta. ¿Qué tabla nueva crearías para responder "lecturas mayores de 35 °C"? (Pista: ya existe una parecida.)

Evidencia 7

Captura de los seis errores y tu respuesta a la pregunta anterior.

Parte 8 · Seguridad, permisos y niveles de acceso

Riesgo o medida	Explicación
Cassandra sin contraseña	La imagen oficial arranca con autenticación desactivada: cualquiera que alcance el puerto 9042 puede leer y borrar todo. Por eso ese puerto **nunca** se hace público.
Solo la API es pública	El navegador y el celular hablan con `servidor.js` (puerto 3000), no con Cassandra. La API decide qué consultas existen.
Sentencias preparadas	Los valores viajan separados del CQL (`?`), lo que evita la inyección de CQL.
Validación en el servidor	`registrar()` rechaza sensores inexistentes y valores fuera de rango, aunque alguien modifique la página.
Roles y permisos	Con autenticación activada, Cassandra maneja roles con `CREATE ROLE` y permisos con `GRANT SELECT` / `MODIFY ON KEYSPACE` (Reto 3).
Datos ficticios y puerto en Private	Usa solo datos simulados y regresa el puerto 3000 a Private al terminar.

Parte 9 · Al terminar

1. Detén el simulador y el servidor con **Ctrl + C**; sal de cqlsh con `exit`.
2. Pon el puerto 3000 en **Private**.
3. Guarda tu trabajo: ícono de **Control de código fuente** → mensaje *"Estación con Cassandra"* → **Confirmar** → **Sincronizar cambios**. Revisa que `node_modules` no se suba (lo excluye `.gitignore`).
4. Detén el codespace: **github.com/codespaces → ... → Stop codespace**.

Tus datos viven en el contenedor de Cassandra dentro del codespace. Si borras el codespace, recréalos con `docker run` (Parte 2), `docker exec -i cassandra cqlsh < cql/esquema.cql` y `npm install` en `app`.

Preguntas de reflexión

1. ¿Por qué en Cassandra se diseñan las tablas a partir de las consultas y en MySQL a partir de las entidades?
2. ¿Qué pasaría si la llave de partición de `lecturas_por_sensor_dia` fuera solo `sensor`, sin `dia`?
3. Una lectura se guarda en 3 o 4 tablas. ¿Qué ganas y qué pierdes con esa duplicación?
4. Explica con tus palabras el error de `ALLOW FILTERING` usando la idea de particiones y nodos.
5. ¿Qué diferencia hay entre el upsert de Cassandra y un INSERT con llave duplicada en MySQL?
6. Compara la implementación de esta práctica con Firestore (servicio administrado) y con IRIS (gestor propio): ¿qué hiciste tú y qué hizo el proveedor en cada caso?
7. ¿Por qué el puerto 9042 nunca debe ser público y el 3000 sí puede serlo?

Retos opcionales (puntos extra)

Reto 1 · Resumen diario

Crea la tabla `resumen_por_dia ((sensor), dia)` con temperatura mínima, máxima y promedio, y un script que la llene leyendo cada partición de `lecturas_por_sensor_dia`. Muéstrala en el tablero.

Reto 2 · Alertas por sensor

Agrega una tabla que responda "las alertas de UN sensor en los últimos días", escríbela en `guardarLectura()` y crea su ruta en la API. Justifica tu llave primaria.

Reto 3 · Autenticación y roles

Activa `PasswordAuthenticator` en `cassandra.yaml` del contenedor, reinícialo, entra con el usuario inicial y crea un rol `app_estacion` que solo pueda leer y escribir en el keyspace `estacion` (`GRANT SELECT` y `GRANT MODIFY`). Conecta `bd.js` con ese rol (`authProvider` del controlador). Documenta cada comando.

Entrega y evaluación

Entreguen un PDF con los integrantes, la dirección de su repositorio (con `cql/esquema.cql` y la carpeta `app` sincronizados), las evidencias 1 a 7 y las respuestas de reflexión.

Criterio	Puntos
Servidor de Cassandra aprovisionado y funcionando (evidencia 1)	10
Keyspace y 5 tablas con llaves correctas; esquema guardado en el repositorio (evidencia 2)	15
Manipulación con CQL: upsert, UPDATE, DELETE, TTL, contador y BATCH (evidencia 3)	15
Simulador y carga histórica funcionando (evidencia 4)	10
API y aplicación web funcionando, incluido el celular (evidencia 5)	15
Observación de particiones y estadísticas (evidencia 6)	10
Laboratorio de errores con explicación (evidencia 7)	10
Preguntas de reflexión	15
Retos opcionales (5 puntos cada uno)	+15
Total	**100**

Solución de problemas

Mensaje o síntoma	Causa y solución
`docker: command not found`	El codespace no usa la imagen por defecto. Crea el codespace desde un repositorio sin carpeta `.devcontainer`.

`Conflict. The container name "/cassandra" is already in use`	Ya lo creaste antes. Enciéndelo con `docker start cassandra` (no repitas `docker run`).
cqlsh: `Connection refused` o `Unable to connect to any servers`	Cassandra sigue arrancando: espera 1 o 2 minutos y revisa `docker logs cassandra`.
El contenedor se detiene solo (`Exited (137)`)	Falta de memoria: confirma `MAX_HEAP_SIZE=1G`; borra el contenedor con `docker rm cassandra` y vuelve a crearlo.
Al reabrir el codespace la app dice "Sin conexión"	`docker ps -a`; si Cassandra está detenido, `docker start cassandra`. Vuelve a ejecutar `node servidor.js` y el simulador.
`Keyspace 'estacion' does not exist` o `unconfigured table`	Falta crear el keyspace o una tabla, o tiene otro nombre: revisa con `DESCRIBE TABLES;`.
`Undefined column name`	Una columna se llama distinto en tu tabla y en el código: compárala con `DESCRIBE TABLE`.
`Cannot find module 'cassandra-driver'`	No ejecutaste `npm install cassandra-driver` dentro de `app`, o estás en otra carpeta.
`EADDRINUSE: address already in use :::3000`	Ya hay un servidor corriendo en otra terminal: ciérralo con Ctrl + C.
El simulador dice que no hay sensores	Inserta los sensores (paso 8) en el keyspace `estacion`.
En el celular aparece el inicio de sesión de GitHub	El puerto 3000 está en Private: ponlo en Public.
La hora en cqlsh no coincide con tu reloj	cqlsh muestra UTC (6 horas más que en el centro de México); la aplicación la convierte a hora local.
Un SELECT se queda esperando en cqlsh	Te faltó el `;` final: escríbelo y presiona Enter.

Práctica 6 de bases de datos NoSQL · Base de datos orientada a columnas con Apache Cassandra en GitHub Codespaces. Datos simulados con fines educativos.